Tech Stack Scout — Components Map

AI-powered tech-stack intelligence for Nikhil's target-account watchlist · MESA C4, Problem Statement 5

Live app: https://tech-stack-scout-kappa.vercel.app · **Repo:** https://github.com/hrishitavijaywargi/tech-stack-scout
Stack: Next.js (TypeScript, Tailwind, Recharts) on Vercel · Supabase Postgres · Gemini Flash via `@google/genai` · 7 live public data sources

The pipeline at a glance

1 · TRIGGER
When
- Vercel Cron weekly, Mon 06:00 UTC
- Vercel Cron daily stale sweep
- Supabase pg_cron, 10 min
- Add / CSV / Refresh now

→

2 · INPUT
What
- `companies` watchlist row
- GitHub, job boards
- DNS, crt.sh
- HN, blog RSS, web search

→

3 · CONTEXT
Knows
- Product profile
- Previous snapshot
- Tech dictionary (~185)
- First-observed dates

→

4 · PROCESSING
Deterministic
- Detect & aggregate stack
- Per-tech confidence
- Activity & fit scores
- Snapshot diff

→

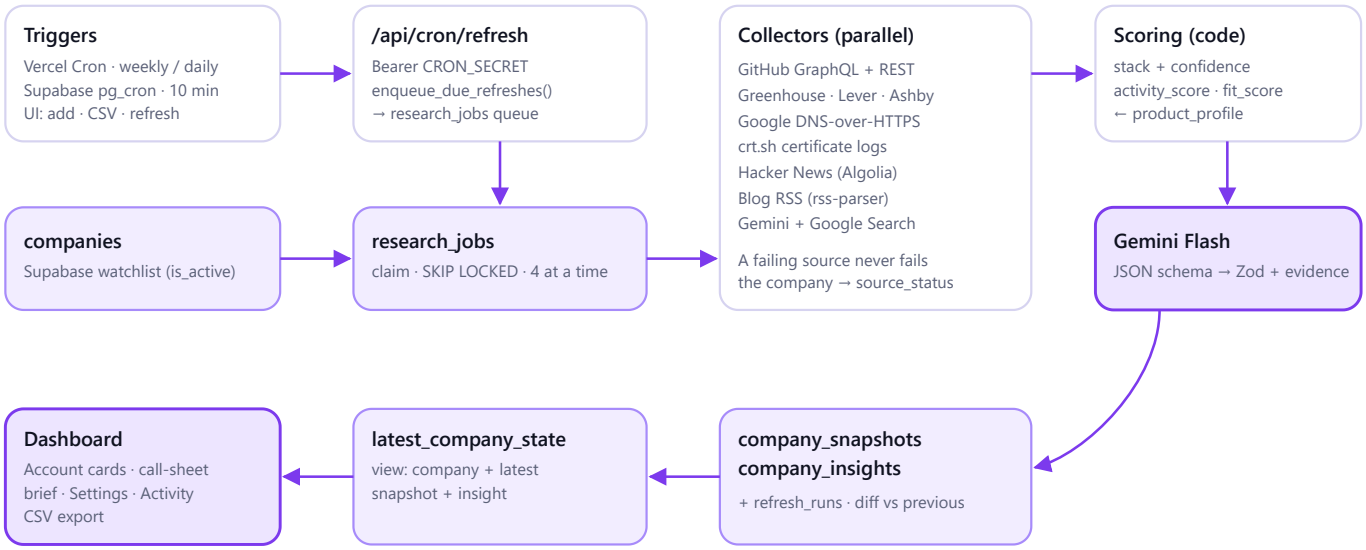
5 · AI
Gemini Flash
- 2-paragraph summary
- Outreach line (<35 words)
- 3 talking points
- Grounded web research

→

6 · OUTPUT
Delivered
- Snapshot + insight rows
- Account cards & brief
- What's changed
- CSV export

Design rule: code decides the facts (which technologies, which scores, what changed); Gemini only writes prose from an ID'd evidence pack, and its answer is validated against that evidence before it is saved. Nothing is invented — missing data is shown as "Not publicly found".

Data flow



1 · Trigger

Component	Implementation	Notes
Weekly refresh	Vercel Cron `0 6 * * 1` → `/api/cron/refresh?source=vercel-weekly`	The freshness guarantee: every active company is re-researched 7 days after its last snapshot.
Daily stale sweep	Vercel Cron `0 7 * * *` → `?source=vercel-daily`	Drains anything older than 7 days so the queue always empties.
Supabase scheduler	pg_cron `*/10 * * * *` → same endpoint (URL + secret in Vault)	Picks up rows added directly in Supabase within 10 min; retries; chained batches.
Manual	Add company · Upload CSV · Refresh now · Research all · Refresh all stale	Same queue and pipeline as scheduled runs. Idempotent: one active job per company.
Security	`Authorization: Bearer CRON_SECRET`, constant-time compare	Vercel Cron sends the header automatically.

2 · Input

Source	Collector	What it contributes
Watchlist	`companies` table	name, domain (required); optional github_org, job board provider/slug, blog_rss_url, notes, is_active. Edited in the UI or Supabase — no code.
GitHub	`collectors/github.ts`	GraphQL: ≤300 non-fork repos with languages in 3 calls; all-time vs recent (180 d) language mix. REST: 52-week commit stats (202 retry), contributors, releases, file trees (Docker, k8s, Helm, Terraform, CI, proto), manifests, workflow contents, Terraform providers, Serverless provider. Microservices heuristic.
Job boards	`collectors/jobs.ts`	Greenhouse / Lever / Ashby public APIs; auto-discovered board saved back; engineering roles; per-technology posting counts; 5 most relevant roles; hiring trend.
DNS	`collectors/dns.ts`	Google DNS-over-HTTPS: CNAME/NS → hosting/CDN, MX → email provider, TXT → ~45 SaaS verification records, SPF includes.
Certificates	`collectors/crtsh.ts`	crt.sh: unexpired subdomains, tooling flags (grafana, jenkins, argocd, k8s, jira…). 25 s timeout, one retry.
Hacker News	`collectors/hn.ts`	Last-12-month engineering stories on the company domain or naming it; top 5 by points.
Engineering blog	`collectors/rss.ts`	Saved or auto-discovered RSS/Atom feed (saved back); 5 latest posts, technical flag.
Web research	`collectors/webResearch.ts`	Gemini + Google Search grounding: talks, migrations, announcements; URLs taken from grounding metadata only. Weak, supplemental evidence.

3 · Context

Context	Where	Used for
Product profile	`product_profile` (Settings page)	Fit score targets (languages, clouds, signals) and outreach relevance.
Tech dictionary	`src/lib/techDictionary.ts`	~185 technologies: category, aliases (golang→Go, GCP→Google Cloud, k8s→Kubernetes), path / dependency / text patterns. Shared by every collector and the UI.
Previous snapshot	`company_snapshots`	What's changed diff, hiring and activity trends.

Evidence history	`research_sources`, `research_findings`	Every source URL and finding with first/last observed dates.
Preferences	`app_settings`	Refresh interval, repos scanned in depth, optional sources.

4 · Processing (deterministic, in code)

Step	Module	Detail
Queue	`lib/jobs/queue.ts`	Atomic claim (FOR UPDATE SKIP LOCKED), 4 concurrent, 240 s per job inside a 300 s function, exponential-backoff retries, rate-limit pause for the whole queue, `refresh_runs` bookkeeping.
Pipeline	`lib/research/pipeline.ts`	Collect (parallel) → store sources/findings → stack → scores → Gemini → snapshot + insight + diff. One pipeline for manual, bulk and scheduled runs.
Stack + confidence	`lib/research/analysis.ts`	One entry per technology with IDs F1…; **high** = 2+ independent source families, **medium** = public code/config or 3+ postings, **low** = single weak signal.
Activity score	`lib/scoring.ts`	Log-scaled: commits 90 d ×30, repos active 30 d ×15, 90 d ×10, contributors ×15, releases 90 d ×15, eng roles ×15.
Fit score	`lib/scoring.ts`	Target languages ×40 (share of recent code, full at 60%), target clouds ×30 (by confidence), target signals ×30 (full at 5). Breakdown shown in the UI.
Language mix	`collectors/github.ts`	By code bytes, each repo capped at 2× the median repo size so a vendored codebase can't dominate.
Change tracking	`diffSnapshots`	Newly observed (≠ newly adopted), not re-observed (≠ stopped using), new postings, hiring change, releases, new repos, language shifts, activity level.

5 · AI (constrained) — Gemini Flash only

Job	How	Guardrails
Evidence pack	`lib/ai/report.ts`	Compact ID'd facts (F tech, M activity, R releases, H/J hiring, D DNS, S subdomains, A articles, W web) + scores + product profile. Never raw API dumps.
Insight	`@google/genai` , JSON `responseSchema`	Two-paragraph summary, outreach line (<35 words) + rationale, 3 talking points with evidence IDs, observation/hypothesis statements.
Validation	Zod + evidence checks	Cited IDs must exist; no technologies outside the evidence; no brackets/placeholders, banned phrases or "you have a problem" claims; the line must name a cited detail. Up to 3 attempts with corrective feedback.
Web research	`googleSearch` grounding tool	Finding URLs come from groundingSupports → groundingChunks; ungrounded lines dropped.
Models & limits	`GEMINI_MODEL` / `GEMINI_FALLBACK_MODEL`	gemini-3.6-flash primary, gemini-3.8-flash fallback on 429/5xx; daily-quota errors pause the queue until reset.
No evidence	Deterministic brief	Gemini is not called; the brief says "insufficient public evidence" and lists next research steps.

6 · Output

Output	Where	Contents
Accounts	`/`	Account cards: logo, fit & activity rings, languages, cloud, eng roles + trend, opener with copy, "new reason to reach out" ribbon; hottest-first sort; card/table views; last run & next schedule.
Call sheet	`/company/[id]`	Sticky header with scores and Copy opener; opener, talking points, summary; stack at a glance; what's changed; tabs for code & activity, hiring, infra, signals, DNS & certs, history, sources & confidence.
Settings	`/settings`	Product profile editor, refresh run history, refresh-all-stale, preferences, removed companies, API connection status.
Activity	`/activity`	Per-job status, attempts, durations, explained errors, retry.
Export	`/api/export`	CSV of every company's latest state (formula-escaped).

Supabase objects

Object	Responsibility
`companies`	The watchlist (domain unique; is_active = on the watchlist).
`company_snapshots`	One row per refresh: languages, recent_languages, infra_signals, activity, hiring, dns, subdomains, hn_stories, blog_posts, web_research, scores + breakdowns, source_status, evidence pack, changes.
`company_insights`	Gemini output per snapshot: summary, outreach_line, outreach_rationale, talking_points, model.
`product_profile`	Single row driving fit and outreach.
`refresh_runs`	Trigger, source, status, processed, errors per processing run.
`research_jobs` / **`research_batches`**	Idempotent queue and bulk progress.
`research_sources` / **`research_findings`**	Evidence store with first/last observed.
`latest_company_state` (view)	Active company + latest snapshot + insight.

Functions	`enqueue_research`, `enqueue_due_refreshes`, `claim_research_jobs` (service role only). RLS on every table; the browser never reads the database.

API routes

Route	Responsibility
`GET /api/cron/refresh`	Scheduler entry: queue due companies, drain queue in background.
`POST /api/companies` · `/import`	Add one / bulk CSV (re-validated) and queue research.
`PATCH/DELETE /api/companies/[id]`	Edit sources, deactivate/restore, delete.
`POST /api/companies/[id]/refresh`	Refresh now.
`POST /api/companies/detect`	Auto-detect GitHub org, job board and blog feed.
`POST /api/research`	Research all / stale / needs attention.
`/api/jobs/progress` · `/retry`	Live progress; retry failed job.
`/api/product-profile` · `/api/settings`	Read/update profile and preferences.
`/api/export` · `/api/health`	CSV export; connection status (no secrets).